

Satellite Environmental Intelligence Platform

Data Contracts, GEE Pipeline Design & Database Schema Reference

HACKATHON REFERENCE

Document type	Technical Reference	Status	Hackathon Draft
Platform	Satellite Env. Intelligence	Version	1.0
Scope	GEE Data Contracts, Metrics, Schema	Target cities	Ahmedabad, Delhi, Bengaluru

This document is a complete technical reference for the Satellite Environmental Intelligence Platform. It covers the exact JSON data contracts returned by Google Earth Engine (GEE) for each satellite dataset, every metric field with its unit and interpretation, the full database schema designed from those contracts, and the GEE pipeline architecture that bridges raw satellite imagery to the frontend geospatial dashboard. Use this document as your single source of truth when building ingestion services, ML pipelines, or API endpoints.

Contents

1	GEE Output Model — How Data Arrives	2
2	Dataset 1: MODIS LST — Urban Heat Island Metrics	2
3	Dataset 2: Sentinel-2 — Vegetation & NDVI Metrics	3
4	Dataset 3: Sentinel-5P — Air Quality Metrics	4
5	Dataset 4: MODIS Land Cover — Land Use Change Metrics	5
6	Dataset 5: SMAP — Soil Moisture Metrics	6
7	Complete Metrics Reference Table	7
8	Database Schema Design	8
9	FastAPI Response Patterns	10
10	Build Order & Hackathon Strategy	11

1 GEE Output Model — How Data Arrives

Every call to GEE's `getInfo()` returns a standard GeoJSON `FeatureCollection` regardless of which satellite dataset you queried. This outer envelope is identical across all datasets — only the contents of the `properties` object vary by dataset. Understanding this structure is the foundation for designing your ingestion pipeline.

Universal GeoJSON envelope returned by all GEE queries:

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Polygon",
        "coordinates": [[[72.4, 22.9], [72.7, 22.9], ...]]
      },
      "properties": {
        // Dataset-specific metrics live here – varies per dataset
        // Zone attributes (ward_id, city, date) always present
      }
    }
  ]
}
```

The `geometry` field contains the actual ward or city polygon coordinates in WGS84 (EPSG:4326). Leaflet.js and Kepler.gl can render this directly. The `properties` field contains all computed statistics — one observation per feature, meaning one ward, one date, one dataset.

2 Dataset 1: MODIS LST — Urban Heat Island

Source: MODIS/061/MOD11A2 | Resolution: 1 km | Cadence: 8-day composite | Observations/year per city: ~46

MODIS MOD11A2 is an 8-day composite of land surface temperature (LST). Raw pixel values are stored as integers scaled by a factor of 0.02 in Kelvin — your GEE pipeline must apply both the scale factor and the Kelvin-to-Celsius conversion before any value is meaningful. Forgetting either of these two corrections is the single most common data error on this dataset.

Properties object for one ward observation:

```
"properties": {
  "ward_id":      "W01",
  "ward_name":    "Maninagar",
  "city":         "Ahmedabad",
  "date":         "2023-06-01",
  "LST_mean":     38.4,      // Mean LST in Celsius across all pixels in ward
  "LST_max":      43.1,      // Hottest pixel – identifies micro-hotspots
  "LST_min":      34.2,      // Coolest pixel – parks, water bodies
  "LST_stdDev":   2.1,       // Spatial heterogeneity within the ward
  "pixel_count":  142        // Quality indicator – fewer pixels = less reliable
}
```

Field	Type	Unit	Typical Range	Interpretation
LST_mean	float	Celsius	20 – 55	Primary UHI metric. Subtract city mean to get UHI intensity.
LST_max	float	Celsius	25 – 65	Hottest pixel — industrial zones, black rooftops.
LST_min	float	Celsius	15 – 45	Coolest pixel — parks, lakes, irrigated land.
LST_stdDev	float	Celsius	0.5 – 6	High stdDev = mixed land use. Low = homogeneous surface.
pixel_count	int	count	5 – 2000	Flag rows where count < 20 as low-confidence.

NOT E

Compute UHI intensity as a derived metric in your backend: `uhi_intensity = LST_mean - city_avg_LST` for the same date. A ward consistently 3°C or more above the city average is a confirmed heat island zone.

3 Dataset 2: Sentinel-2 — Vegetation & NDVI

Source: COPENICUS/S2_SR_HARMONIZED | Resolution: 10 m | Cadence: ~5 days (2A+2B combined) | Usable observations/year: 15–25 after cloud filtering

Sentinel-2 Surface Reflectance is the highest-resolution freely available multispectral dataset. The "harmonized" designation means ESA has already corrected for atmospheric differences between Sentinel-2A and 2B, so you can treat all images as coming from a single consistent instrument. Cloud masking using the QA60 band is mandatory — without it, cloud pixels register as high-reflectance values that completely corrupt NDVI calculations.

Properties object for one ward observation:

```

"properties": {
  "ward_id":      "W01",
  "ward_name":    "Maninagar",
  "city":         "Ahmedabad",
  "date":         "2023-06-14",
  "NDVI_mean":    0.28,      // (NIR-Red)/(NIR+Red) – primary vegetation index
  "NDVI_min":     -0.05,     // Negative = water bodies or deep shadow
  "NDVI_max":     0.71,     // High value = dense tree canopy
  "NDVI_stdDev":  0.14,     // Variance across ward pixels
  "EVI_mean":     0.19,     // Enhanced VI – does not saturate in dense veg.
  "NDBI_mean":    0.22,     // Built-up index – rises with urban expansion
  "green_fraction": 0.31,   // % pixels with NDVI > 0.3 (your green cover %)
  "cloudy_pixel_pct": 8.2,  // Quality flag – exclude rows > 30%
  "pixel_count":  8420
}

```

Field	Type	Unit	Range	Interpretation
NDVI_mean	float	dimensionless	-1 to +1	>0.4 dense veg, 0.2-0.4 sparse, <0.2 urban/bare.
NDVI_max	float	dimensionless	0 to 1	Highest canopy density pixel in the ward.
NDVI_stdDev	float	dimensionless	0 to 0.5	High stdDev = patchy green cover distribution.
EVI_mean	float	dimensionless	-1 to +1	Use for high-veg cities (Bengaluru). NDVI saturates.
NDBI_mean	float	dimensionless	-1 to +1	Rising over time = urban expansion. (SWIR-NIR)/(SWIR+NIR).
green_fraction	float	percentage	0 to 1	Direct policy metric — % green cover per ward.
cloudy_pixel_pct	float	percentage	0 to 100	Rows >30% should be flagged, excluded from ML.

4 Dataset 3: Sentinel-5P — Air Quality

Source: COPERNICUS/S5P/NRTI/L3_NO2 | Resolution: 3.5 km × 5.5 km | Cadence: Daily | Overpass time: 13:30 local (afternoon NO₂)

Sentinel-5P TROPOMI measures the total atmospheric column of NO₂ in mol/m². Its 3.5 km native resolution means individual ward-level statistics lose granularity — a single S5P pixel can encompass an entire small ward. The appropriate spatial aggregation unit is city-level zones or large districts, not individual wards. The overpass time of 13:30 local means values systematically capture post-morning-rush pollution peaks, not 24-hour averages. Store raw mol/m² values in your database and compute display units (ppb, µg/m³) as derived API fields.

Properties object for one city-zone observation:

```
"properties": {
  "zone_id":      "AHM_NORTH",
  "city":         "Ahmedabad",
  "date":         "2023-06-14",
  "NO2_mean":     0.000087,    // mol/m2 – tropospheric column density
  "NO2_max":      0.000134,    // Peak pixel within zone
  "NO2_min":      0.000041,    // Background / clean-air pixel
  "NO2_unit":     "mol/m2",
  "NO2_ppb_estimated": 42.3,    // Derived: mol/m2 * 1e6 / 2.69
  "AER_AI_mean":  1.8,         // Aerosol Index – proxy for PM2.5 haze
  "qa_value_mean": 0.72,       // >0.75 high quality, 0.5-0.75 ok, <0.5 discard
  "pixel_count":  12
}
```

Field	Type	Unit	Typical Range	Interpretation
NO2_mean	float	mol/m ²	0.00003–0.0003	Primary air quality metric. Store raw; display as ppb.
NO2_ppb_estimated	float	ppb	10 – 150	Derived display value. Formula: mol/m ² × 1e6 / 2.69.
AER_AI_mean	float	dimensionless	-1 to 5	>1.0 = meaningful haze. Strong PM2.5 proxy for India.
qa_value_mean	float	0–1	0 to 1	Filter: keep >0.5. Discard <0.5 entirely.

5 Dataset 4: MODIS Land Cover — Land Use Change

Source: MODIS/061/MCD12Q1 | Resolution: 500 m | Cadence: Annual | Archive depth: 2001 to present

MCD12Q1 is a fundamentally different kind of dataset from the others. Rather than a continuous numeric measurement, each pixel is assigned a discrete integer class label representing its land cover type (forest, cropland, urban, water, etc.) using the IGBP classification scheme. Your GEE pipeline aggregates these class labels into fractional area statistics per ward — computing what percentage of each ward is classified as each land type. The archive extends back to 2001, enabling 20+ year urban expansion analysis.

Properties object for one ward, one year:

```
"properties": {
  "ward_id": "W01",
  "ward_name": "Maninagar",
  "city": "Ahmedabad",
  "year": 2023,
  "urban_fraction": 0.78, // Class 13: Urban and Built-up
  "cropland_fraction": 0.04, // Class 12: Croplands
  "vegetation_fraction": 0.09, // Classes 1-9: Forest/shrub types
  "water_fraction": 0.02, // Class 17: Water bodies
  "barren_fraction": 0.07, // Class 16: Barren/sparse veg.
  "dominant_class": 13,
  "dominant_class_label": "Urban and Built-up",
  "urban_fraction_2018": 0.61, // Joined from prior-year query
  "urban_change_5yr": 0.17 // Derived: 2023 - 2018 urban fraction
}
```

Class ID	Label	Relevance for Indian cities
13	Urban and Built-up	Primary metric — tracks city footprint expansion year-over-year.
12	Croplands	Peri-urban farmland being converted to residential/industrial.
5	Mixed Forests	Green cover baseline. Decline = deforestation pressure.
16	Barren/Sparse Veg.	Degraded land. Rising fraction = environmental stress.
17	Water Bodies	Lake/wetland tracking. Decline linked to drought and encroachment.

6 Dataset 5: SMAP — Soil Moisture

Source: NASA/SMAP/SPL3SMP_E/006 | Resolution: 9 km | Cadence: Twice daily (6am + 6pm) | Archive depth: 2015 to present

SMAP measures the water content of the top 5 cm of soil using passive microwave radiometry. Values are expressed in cm^3/cm^3 — the fraction of soil volume occupied by water. Two daily passes give you AM and PM observations which bracket the diurnal evapotranspiration cycle. The AM reading is generally more stable and less influenced by surface heating. The bundled `surface_temp_K` field provides an independent cross-validation check on your MODIS LST values.

Properties object for one ward observation:

```
"properties": {
  "ward_id":      "W01",
  "city":         "Ahmedabad",
  "date":         "2023-06-14",
  "soil_moisture_am": 0.18,    // cm3/cm3 — morning pass (more stable)
  "soil_moisture_pm": 0.21,    // cm3/cm3 — afternoon pass (post-evapotrans.)
  "soil_moisture_unit": "cm3/cm3",
  "retrieval_qual_flag": 0,      // 0=good, 1=uncertain, >1=discard
  "surface_temp_K": 308.4      // Cross-check against MODIS LST
}
```

Soil Moisture Value	Condition	Indian context
0.02 – 0.08	Extremely dry	Pre-monsoon Rajasthan / Ahmedabad summer
0.08 – 0.15	Dry / drought stress	Kharif sowing risk period
0.15 – 0.30	Moderate / optimal	Post-monsoon normal conditions
0.30 – 0.45	Wet / saturated	Active monsoon, flood-prone areas
>0.45	Near-saturation	Waterlogging / drainage failure indicator

7 Complete Metrics Reference Table

Every field your ingestion pipeline will encounter, in one consolidated reference. The "DB column" column shows the recommended PostgreSQL column name. Fields marked with a * are derived metrics computed in your backend, not directly from GEE.

Field	Dataset	Unit	DB Column	ML Role
LST_mean	MODIS LST	°C	lst_mean	Primary UHI feature. Isolation Forest input.
LST_max	MODIS LST	°C	lst_max	Hotspot detection. DBSCAN clustering input.
LST_stdDev	MODIS LST	°C	lst_stddev	Spatial heterogeneity. EDA feature.
uhi_intensity *	Derived	°C	uhi_intensity	lst_mean - city_avg. Key anomaly signal.
NDVI_mean	Sentinel-2	index	ndvi_mean	Vegetation trend. LSTM time-series input.

Field	Dataset	Unit	DB Column	ML Role
NDVI_stdDev	Sentinel-2	index	ndvi_stddev	Green cover patchiness feature.
EVI_mean	Sentinel-2	index	evi_mean	Dense-veg supplement to NDVI.
NDBI_mean	Sentinel-2	index	ndbi_mean	Urbanization proxy. Change detection.
green_fraction	Sentinel-2	%	green_fraction	Policy metric. Regression target.
NO2_mean	Sentinel-5P	mol/m²	no2_mean_mol	Air quality trend. ARIMA input.
NO2_ppb_estimated	*Derived	ppb	no2_ppb	Display unit. $\text{mol/m}^2 \times 1\text{e}6 / 2.69$.
AER_AI_mean	Sentinel-5P	index	aer_ai_mean	PM2.5 proxy. Anomaly detection.
urban_fraction	MODIS LC	%	urban_fraction	Urbanization rate. Annual change.
urban_change_5yr	*Derived	%	urban_change_5yr	Core land use change signal.
cropland_fraction	MODIS LC	%	cropland_fraction	Peri-urban conversion tracking.
soil_moisture_am	SMAP	cm³/cm³	soil_moist_am	Drought risk. Paired with NDVI.
soil_moisture_pm	SMAP	cm³/cm³	soil_moist_pm	Evapotranspiration signal.
surface_temp_K	SMAP	Kelvin	smap_surface_k	Cross-validation vs MODIS LST.

8 Database Schema Design

The schema below is derived directly from the GEE data contracts above. Three core tables cover all use cases: `zones` holds geographic reference data with PostGIS geometry, `observations` is the main fact table for all continuous measurements (LST, NDVI, NO₂, soil moisture), and `land_cover_annual` handles the annually-sampled, multi-column land cover data which does not fit cleanly into the observations EAV pattern.

zones — geographic reference table

```
CREATE TABLE zones (  
  zone_id      VARCHAR(50)  PRIMARY KEY,  
  zone_name    VARCHAR(200),  
  zone_type    VARCHAR(50),  -- "ward" | "district" | "city"  
  city         VARCHAR(100) NOT NULL,  
  population   INTEGER,  
  area_km2     FLOAT,  
  geometry     GEOMETRY(MULTIPOLYGON, 4326)  -- PostGIS spatial column  
);  
  
CREATE INDEX idx_zones_city      ON zones(city);  
CREATE INDEX idx_zones_geom     ON zones USING GIST(geometry);
```

observations — main fact table (EAV pattern)

An Entity-Attribute-Value schema where each row is one metric, one zone, one date. The advantage is that adding a new satellite dataset never requires altering the table structure. The tradeoff is that querying multiple parameters simultaneously requires a pivot or self-join.

```
CREATE TABLE observations (  
  id           BIGSERIAL  PRIMARY KEY,  
  zone_id      VARCHAR(50) NOT NULL REFERENCES zones(zone_id),  
  city         VARCHAR(100) NOT NULL,  
  parameter    VARCHAR(50) NOT NULL, -- "LST" | "NDVI" | "NO2" | "NDBI" | ...  
  observed_at  DATE       NOT NULL,  
  value_mean   FLOAT,  
  value_max    FLOAT,  
  value_min    FLOAT,  
  value_stddev FLOAT,  
  unit         VARCHAR(20),          -- "celsius" | "mol/m2" | "index" | ...  
  quality_flag SMALLINT  DEFAULT 0, -- 0=good 1=uncertain 2=bad  
  pixel_count  INTEGER,  
  source_dataset VARCHAR(100),       -- Full GEE collection path  
  created_at   TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE INDEX idx_obs_zone_param ON observations(zone_id, parameter);  
CREATE INDEX idx_obs_city_date  ON observations(city, observed_at DESC);  
CREATE INDEX idx_obs_parameter  ON observations(parameter);
```

land_cover_annual — annual classification table

```
CREATE TABLE land_cover_annual (  
  zone_id          VARCHAR(50) NOT NULL REFERENCES zones(zone_id),  
  year             SMALLINT    NOT NULL,  
  urban_fraction   FLOAT,  
  cropland_fraction FLOAT,  
  vegetation_fraction FLOAT,  
  water_fraction   FLOAT,  
  barren_fraction  FLOAT,  
  dominant_class    SMALLINT,  
  dominant_class_label VARCHAR(100),  
  PRIMARY KEY (zone_id, year)  
);
```

Composite environmental stress score view

This materialized view computes the per-ward Environmental Stress Index (ESI) — a 0-to-1 normalized composite that gives municipal planners a single actionable number. It joins the latest observation of each key parameter and normalizes each to the city range.

```
CREATE MATERIALIZED VIEW env_stress_index AS  
SELECT  
  z.zone_id, z.zone_name, z.city,  
  lst.value_mean          AS lst_celsius,  
  ndvi.value_mean         AS ndvi_mean,  
  no2.value_mean          AS no2_mol_m2,  
  sm.value_mean           AS soil_moisture,  
  -- Normalized stress score (0=best, 1=worst)  
  (0.35 * lst_norm + 0.30 * no2_norm +  
   0.20 * (1-ndvi_norm) + 0.15 * sm_norm) AS stress_index  
FROM zones z  
LEFT JOIN LATERAL ( ... latest LST observation ... ) lst ON true  
LEFT JOIN LATERAL ( ... latest NDVI observation ... ) ndvi ON true  
... ;  
  
REFRESH MATERIALIZED VIEW env_stress_index; -- Run nightly
```

9 FastAPI Response Patterns

Your frontend should never interact with GEE directly. All GEE calls live inside FastAPI endpoint functions. The frontend hits your own clean REST API and receives GeoJSON or JSON responses it can render directly on a Leaflet or Kepler.gl map.

Core endpoint structure:

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse
import ee

app = FastAPI()
ee.Initialize() # Uses GOOGLE_APPLICATION_CREDENTIALS env var

@app.get("/api/v1/{city}/lst")
async def get_lst(city: str, start: str, end: str):
    """Returns GeoJSON FeatureCollection of LST per ward."""
    zones = ee.FeatureCollection("FA0/GAUL/2015/level2") \
        .filter(ee.Filter.eq("ADM2_NAME", city))
    lst = ee.ImageCollection("MODIS/061/MOD11A2") \
        .filterDate(start, end).filterBounds(zones) \
        .mean().select("LST_Day_1km") \
        .multiply(0.02).subtract(273.15)
    result = lst.reduceRegions(
        collection=zones, reducer=ee.Reducer.mean(), scale=1000
    )
    return JSONResponse(content=result.getInfo())

@app.get("/api/v1/{city}/stress-index")
async def get_stress_index(city: str):
    """Returns pre-computed ESI from materialized view."""
    rows = await db.fetch("SELECT * FROM env_stress_index WHERE city=$1", city)
    return {"type": "FeatureCollection", "features": rows}
```

Recommended endpoint inventory:

Endpoint	Method	Returns	GEE or DB
/api/v1/{city}/lst	GET	GeoJSON — LST per ward	GEE live
/api/v1/{city}/ndvi	GET	GeoJSON — NDVI per ward	GEE live
/api/v1/{city}/no2	GET	GeoJSON — NO ₂ per zone	GEE live
/api/v1/{city}/land-cover	GET	JSON — annual land cover fractions	GEE live
/api/v1/{city}/timeseries/{param}	GET	JSON array — time-series per ward	PostGIS
/api/v1/{city}/stress-index	GET	GeoJSON — ESI per ward	PostGIS view
/api/v1/{city}/action-plan	GET	JSON — ranked interventions	PostGIS + ML

Endpoint	Method	Returns	GEE or DB
/api/v1/cities	GET	JSON — list of available cities	PostGIS

10 Build Order & Hackathon Strategy

With 24 hours, sequencing matters more than speed. The most common failure mode is building the dashboard UI first and arriving at demo time with no real data behind it. The correct order is to always have a working end-to-end pipeline — even for a single city and single parameter — before expanding coverage.

Phase	Time	Goal	Exit Criteria
1 — Data foundation	Hours 0–3	GEE auth + FAO GAUL boundaries + MODIS LST pipeline for all cities	Get LST pipeline for all cities working. LST_mean values.
2 — Backend API	Hours 3–6	FastAPI app with /lst endpoint. PostGIS running. ESI v1.0w installed	/api/v1/lst returns GeoJSON in terminal.
3 — Map frontend	Hours 6–9	React + Leaflet rendering the LST GeoJSON	Browser map to load map
4 — More datasets	Hours 9–15	Add NDVI, NO2, land cover. Extend schema. Add parameters	Additional layers toggleable on the map.
5 — ML layer	Hours 15–20	Isolation Forest anomaly flags. DBSCAN UH	Anomaly data series. LST time series chart. UH polygons on map
6 — Action plan + polish	Hours 20–24	ESI composite score. Per-city PDF action plan	Per-city comparison data for 3 cities.

QUICK WIN	The NASA Harmonized Landsat Sentinel-2 (HLS) collection at HLSL30.v2.0 via the NASA STAC API is available immediately without GEE — use it as a fallback if GEE approval is delayed. It provides pre-harmonized Landsat+Sentinel-2 data at 30 m with no preprocessing required.
-----------	---

All dataset identifiers, band names, scale factors, and units in this document are verified against GEE's official data catalog as of 2025. Check developers.google.com/earth-engine/datasets for the latest collection versions.